



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 07

NOMBRE COMPLETO: Lechuga Castillo Shareny Ixchel

N° de Cuenta: 319004252

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 04

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 16-10-2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

- 1.-Agregar movimiento con teclado al helicóptero hacia adelante y atrás.
- 2.-Crear luz spotlight de helicóptero de color amarilla que apunte hacia el piso y se mueva con el helicóptero
- 3.- Añadir en el escenario 1 modelo de lámpara texturizada (lo usarán en su proyecto final) y crearle luz puntual blanca

El código que obtenemos es el siguiente:

```
Practica7x64 (Ámbito global)
28 #include "DirectionalLight.h"
29 #include "PointLight.h"
30 #include "SpotLight.h"
31 #include "Material.h"
32 const float toRadians = 3.14159265f / 180.0f;
33
34 Window mainWindow;
35 std::vector<Mesh*> meshList;
36 std::vector<Shader> shaderList;
37
38 Camera camera;
39
40 Texture brickTexture;
41 Texture dirtTexture;
42 Texture plainTexture;
43 Texture pisoTexture;
44 Texture AgaveTexture;
45 Texture Lamp;
46
47 Model Kitt_M;
48 Model Llanta_M;
49 Model Blackhawk_M;
50 Model Helicoptero;
51 Model Helice1;
52 Model Helice2;
53 Model Lampara;
54 Skybox skybox;
55
56 //materiales
57 Material Material_brillante;
58 Material Material_opaco;
59
60
61 //Sphere cabeza = Sphere(0.5, 20, 20);
62 GLfloat deltaTime = 0.0f;
63 GLfloat lastTime = 0.0f;
64 static double limitFPS = 1.0 / 60.0;
65
66 // luz direccional
67 DirectionalLight mainLight;
68 //para declarar varias luces de tipo pointlight
69 PointLight pointLights[MAX_POINT_LIGHTS];
70 SpotLight spotLights[MAX_SPOT_LIGHTS];
71
72 // Vertex Shader
73 static const char* vShader = "shaders/shader_light.vert";
74
75 // Fragment Shader
88 % No se encontraron problemas.
Lista de errores Salida
```

```

191 int main()
192 {
193     mainWindow = Window(1366, 768); // 1280, 1024 or 1024, 768
194     mainWindow.Initialise();
195
196     CreateObjects();
197     CreateShaders();
198
199     camera = Camera(glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 1.0f, 0.0f), -60.0f, 0.0f, 0.3f, 0.5f);
200
201     brickTexture = Texture("Textures/brick.png");
202     brickTexture.LoadTextureA();
203     dirtTexture = Texture("Textures/dirt.png");
204     dirtTexture.LoadTextureA();
205     plainTexture = Texture("Textures/plain.png");
206     plainTexture.LoadTextureA();
207     pisoTexture = Texture("Textures/piso.tga");
208     pisoTexture.LoadTextureA();
209     AgaveTexture = Texture("Textures/Agave.tga");
210     AgaveTexture.LoadTextureA();
211     Lamp = Texture("Textures/lamp.tga");
212     Lamp.LoadTextureA();
213
214     Kitt_M = Model();
215     Kitt_M.LoadModel("Models/kitt_optimizado.obj");
216     Llanta_M = Model();
217     Llanta_M.LoadModel("Models/Llanta_optimizada.obj");
218     Blackhawk_M = Model();
219     Blackhawk_M.LoadModel("Models/uh60.obj");
220
221     Helicoptero = Model();
222     Helicoptero.LoadModel("Models/Helicoptero.obj");
223     Helice1 = Model();
224     Helice1.LoadModel("Models/Helice1.obj");
225     Helice2 = Model();
226     Helice2.LoadModel("Models/Helice2.obj");
227
228     Lampara = Model();
229     Lampara.LoadModel("Models/Lampara.obj");
230
231     std::vector<std::string> skyboxFaces;
232     skyboxFaces.push_back("Textures/Skybox/cupertin-lake_rt.tga");
233     skyboxFaces.push_back("Textures/Skybox/cupertin-lake_lf.tga");
234     skyboxFaces.push_back("Textures/Skybox/cupertin-lake_dn.tga");
235     skyboxFaces.push_back("Textures/Skybox/cupertin-lake_up.tga");
236     skyboxFaces.push_back("Textures/Skybox/cupertin-lake_bk.tga");
237     skyboxFaces.push_back("Textures/Skybox/cupertin-lake_ft.tga");
238
239     skybox = Skybox(skyboxFaces);
240
241     Material_brillante = Material(4.0f, 256);
242     Material_opaco = Material(0.3f, 4);
243
244     //luz direccional, sólo 1 y siempre debe de existir
245     mainLight = DirectionalLight(1.0f, 1.0f, 1.0f,
246         0.3f, 0.3f,
247         0.0f, 0.0f, -1.0f);
248     //contador de luces puntuales
249     unsigned int pointLightCount = 0;
250     //Declaración de primer luz puntual
251     pointLights[0] = PointLight(1.0f, 1.0f, 1.0f,
252         0.0f, 1.0f,
253         6.0f, 1.5f, 0.0f,
254         0.1f, 0.07f, 0.05f);
255     pointLightCount++;
256
257     unsigned int spotLightCount = 0;
258     //linterna
259
260     //luz fija
261     spotLights[0] = SpotLight(1.0f, 1.0f, 0.0f, // Color Azul
262         2.5f, 5.0f, // componentes ambiental y difusa
263         0.0f, 4.2f, 3.5f, // Posición inicial
264         0.0f, -1.0f, 0.0f, // Dirección hacia donde apunta la luz
265         1.0f, 0.03f, 0.05f, // atenuaciones
266         20.0f); // Ángulo de apertura
267     spotLightCount++;
268
269     //se crean mas luces puntuales y spotlight
270
271     GLuint uniformProjection = 0, uniformModel = 0, uniformView = 0, uniformEyePosition = 0,
272         uniformSpecularIntensity = 0, uniformShininess = 0;
273     GLuint uniformColor = 0;
274     glm::mat4 projection = glm::perspective(45.0f, (GLfloat)mainWindow.getBufferWidth() / mainWindow.getBufferHeight(), 0.1f, 1000.0f);
275     ///Loop mientras no se cierra la ventana
276     while (!mainWindow.getShouldClose())
277     {
278         GLfloat now = glfwGetTime();
279         deltaTime = now - lastTime;
280

```

```

277 while (!mainWindow.getShouldClose())
278 {
279     GLfloat now = glfwGetTime();
280     deltaTime = now - lastTime;
281     deltaTime += (now - lastTime) / limitFPS;
282     lastTime = now;
283
284     //Recibir eventos del usuario
285     glfwPollEvents();
286     camera.keyControl(mainWindow.getKeys(), deltaTime);
287     camera.mouseControl(mainWindow.getXChange(), mainWindow.getYChange());
288
289     // Clear the window
290     glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
291     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
292     skybox.DrawSkybox(camera.calculateViewMatrix(), projection);
293     shaderList[0].UseShader();
294     uniformModel = shaderList[0].GetModelLocation();
295     uniformProjection = shaderList[0].GetProjectionLocation();
296     uniformView = shaderList[0].GetViewLocation();
297     uniformEyePosition = shaderList[0].GetEyePositionLocation();
298     uniformColor = shaderList[0].getColorLocation();
299
300     //información en el shader de intensidad especular y brillo
301     uniformSpecularIntensity = shaderList[0].GetSpecularIntensityLocation();
302     uniformShininess = shaderList[0].GetShininessLocation();
303
304     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
305     glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
306     glUniform3fv(uniformEyePosition, camera.getCameraPosition().x, camera.getCameraPosition().y, camera.getCameraPosition().z);
307     glm::mat4 model(1.0);
308     glm::mat4 modelaux(1.0);
309     glm::mat4 modelaux2(1.0);
310     // trasladar la luz del faro segun el Helicoptero
311     // 3. Actualizar la posición y dirección del 'SpotLight'
312     spotlights[0].setFlash(glm::vec3(0.0f, 4.2f, 3.5f+ mainWindow.getDesplazamiento()), glm::vec3(0.0f, -1.0f, 0.0f));
313     //información al shader de fuentes de iluminación
314     shaderList[0].SetDirectionalLight(&mainLight);
315     shaderList[0].SetPointLights(pointLights, pointLightCount);
316     shaderList[0].SetSpotLights(spotLights, spotLightCount);
317
318     glm::vec3 color = glm::vec3(1.0f, 1.0f, 1.0f);
319
320     model = glm::mat4(1.0);
321     model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
322     model = glm::scale(model, glm::vec3(30.0f, 1.0f, 30.0f));
323     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
324     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
325
326     pisoTexture.UseTexture();
327     Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
328
329     meshList[2]->RenderMesh();
330
331     model = glm::mat4(1.0);
332     model = glm::translate(model, glm::vec3(0.0f, 5.0f, 6+mainWindow.getDesplazamiento()));
333     modelaux = model;
334     model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
335     /*model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
336     model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));*/
337     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
338     Helicoptero.RenderModel();
339     //Helice1
340     model = modelaux;
341     model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 1.0f, 0.0f));
342     model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
343     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
344     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
345     Helice1.RenderModel();
346     //Helice2
347     model = modelaux;
348     model = glm::translate(model, glm::vec3(0.15, 0.65f, 4.9f));
349     model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(1.0f, 0.0f, 0.0f));
350     model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
351     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
352     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
353     Helice2.RenderModel();
354     //Lampara
355     model = glm::mat4(1.0);
356     model = glm::translate(model, glm::vec3(6.0f, -1.0f, 0.0f));
357     model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
358     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
359     Lampara.RenderModel();

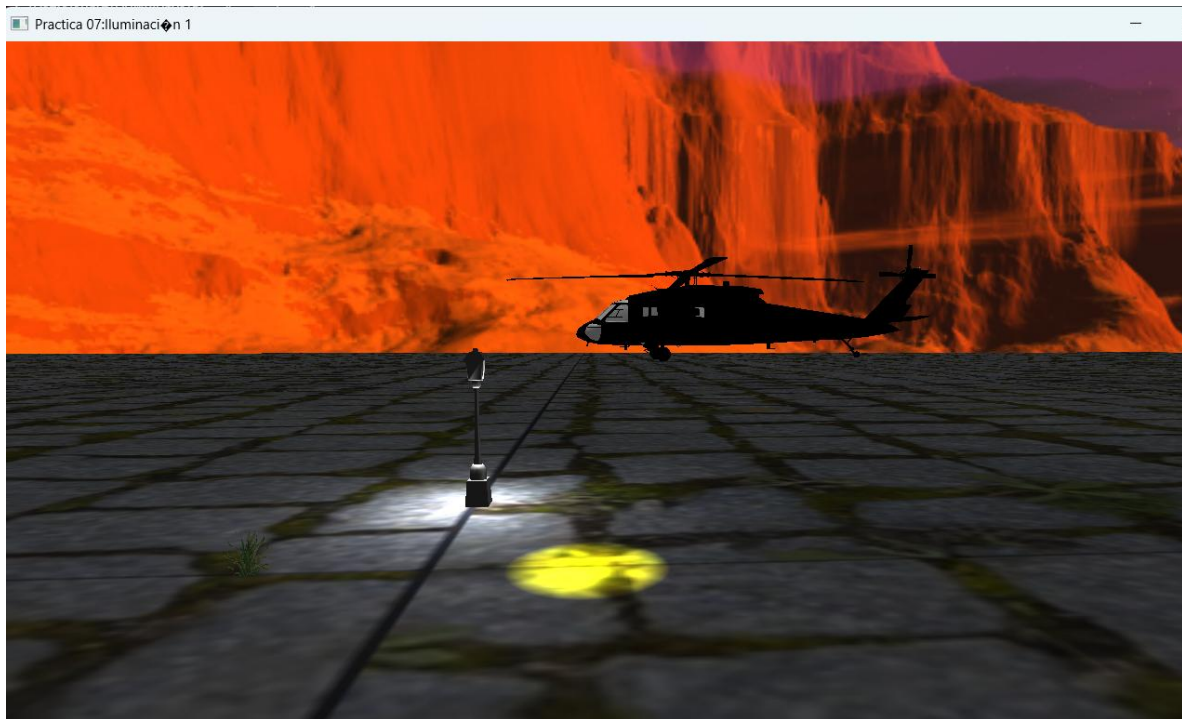
```

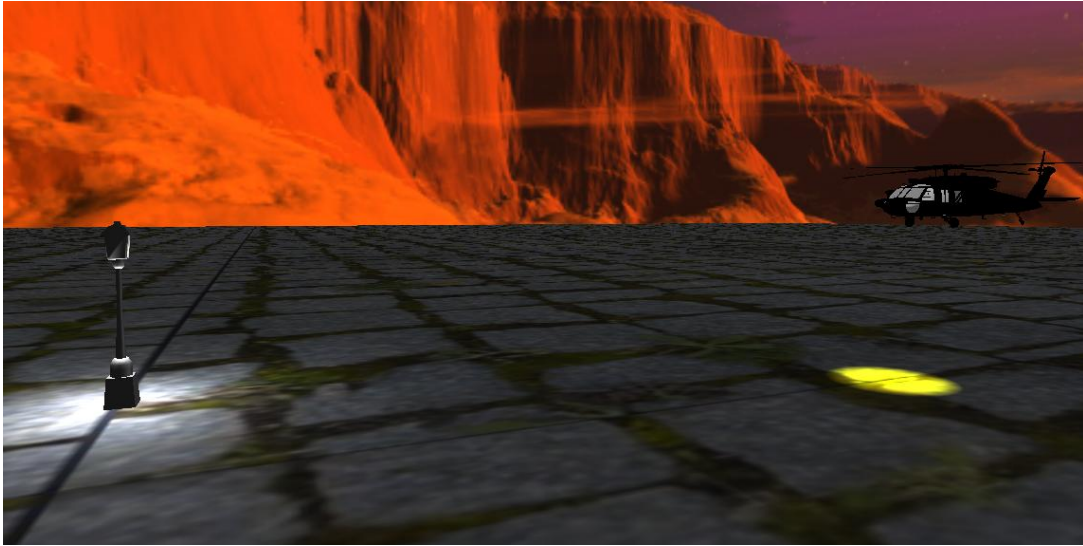
```

361 //Agave ¿qué sucede si lo renderizan antes del coche y el helicóptero?
362 model = glm::mat4(1.0);
363 model = glm::translate(model, glm::vec3(0.0f, -0.5f, -4.0f));
364 //model = glm::scale(model, glm::vec3(4.0f, 4.0f, 4.0f));
365 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
366
367
368 //blending: transparencia o traslucidez
369 glEnable(GL_BLEND);
370 glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
371 AgaveTexture.UseTexture();
372 Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
373 meshList[3]->RenderMesh();
374 glDisable(GL_BLEND);
375
376 glUseProgram(0);
377
378 mainWindow.swapBuffers();
379 }
380
381 return 0;
382 }

```

El ejecutable es el siguiente:





Problemas:

Durante esta práctica, el único problema que surgió es que inicialmente al importar la lámpara no funcionaban las texturas, pero esto se debió a que la textura estaba en formato .png al ponerlo en .tga y cambiar el .mlt del objeto se resolvió de manera rápida.

Conclusión:

- a. Los ejercicios del reporte: No fue complicado dejando de lado el problema con la lámpara, aún se me dificulta usar blender para separar las hélices, pero creo que lo podré lograr.
- b. Comentarios generales: Me gustó mucho esta explicación ya que hizo apuntes y fue con mucha más calma, el tiempo que se tomó fue el necesario para entender cada parte del código.
- c. Conclusión: En general estuvo "fácil" como siempre he dicho con la práctica podré hacer todo mejor y más rápido.

Bibliografía en formato APA

LearnOpenGL - Basic Lighting. (s. f.).
<https://learnopengl.com/Lighting/BasicLighting>